

Homework 4

Name: Gatman Vitalie
Date: October 20, 2026
Group: FAF-263

Problem 4.1

1. I use Erastotene stiv to find all prime numbers up to n
2. Using a linear search i find the values of p and q
   Note: Binary search would had ben faster with $O(n \log n)$ time insted of $O(n^2)$ but for this problem is enough
3. I compute the Totient $\phi(n) = (q-1)(p-1)$
4. I find the value d via linear search
5. I compute the decrypted result using $m = c^d \mod n$ where c is the inputed variable an m is teh outputs
6. Result: $X = 20$, $Y = 10$, $Z = 11$
7. Code in C++:

```cpp
#include <iostream>
#include <vector>

using namespace std;

// Expoenitnation
long long modPow(long long base, long long exp, long long mod)
   {
    long long res = 1;
    base %= mod;
    while (exp > 0) {
        if (exp % 2 == 1) res = (res * base) % mod;
        base = (base * base) % mod;
        exp /= 2;
    }
    return res;
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int e = 84733;
    int n = 89951;

    // Sieve of Eratosthenes
    vector<bool> stiv(n + 1, false);
    vector<int> primes;

    for (int i = 2; i <= n; i++) {
        if (!stiv[i]) {
            primes.push_back(i);
            for (int j = i * 2; j <= n; j += i) {
                stiv[j] = true;
            }
        }
    }

```



```
    // Find prime factors p and q
    long long p = 0, q = 0;
    int len = primes.size();
    for (int i = len - 1; i >= 0; i--) {
        for (int j = 0; j < len; j++) {
            long long prod = (long long)primes[i] * primes[j];
            if (prod == n) {
                p = primes[j];
                q = primes[i];
                i = -1; // break outer loop
                break;
            }
            else if (prod > n) {
                break;
            }
        }
    }

    long long phi = (p - 1) * (q - 1);

    // Find private exponent d
    long long d = -1;
    for (long long i = 1; i <= phi; i++) {
        if ((i * e) % phi == 1) {
            d = i;
            break;
        }
    }

    long long c;
    while (cin >> c) {
        cout << modPow(c, d, n) << "\n";
    }

    return 0;
}
```

Problem 4.3

By Bézout's Identity

a) *Proof.* Let $d = \gcd(a,b) = ax_0 + by_0$ and $g = \gcd(ka, kb)$. Then $kd = ka\,x_0 + kb\,y_0$, so $g \mid kd$. Also $d \mid a,\ d \mid b \Rightarrow kd \mid ka,\ kd \mid kb \Rightarrow kd \mid g$. Hence $g = kd$. □

b) *Proof.*

$$\gcd(a,b) = ax_1 + by_1 = 1$$

$$\gcd(a,c) = ax_2 + cy_2 = 1$$

$$(ax_1+by_1)(ax_2+cy_2) = a^2x_1x_2+acx_1y_2+abx_2y_1+bcy_1y_2 = a\underbrace{(ax_1x_2 + cx_1y_2 + bx_2y_1)}_{P} + bc\underbrace{(y_1y_2)}_{Q}$$

Because there exist a combination of integers P, Q such that $aP + bcQ = 1$ them $\gcd(a, bc) = 1$ □

c) *Proof.* Let $ax + by = 1$. Then $acx + bcy = c$. Since $a \mid ac$ and $a \mid bc$, we get $a \mid c$. □

Problem 4.4

a) $21 \mid 48 \implies s = 0,\ t = 4$

b)

$$\begin{aligned}
174 &= 3 \cdot 48 + 30 \\
48 &= 1 \cdot 30 + 18 \\
30 &= 1 \cdot 18 + 12 \\
18 &= 1 \cdot 12 + 6 \\
12 &= 2 \cdot 6 + 0
\end{aligned}$$

$$\begin{aligned}
6 &= 18 - 1 \cdot 12 \\
12 &= 30 - 1 \cdot 18 \\
18 &= 48 - 1 \cdot 30 \\
30 &= 174 - 3 \cdot 48
\end{aligned}$$

$$\begin{aligned}
6 &= 18 - 1 \cdot (30 - 1 \cdot 18) \\
&= 2 \cdot 18 - 1 \cdot 30 \\
&= 2 \cdot (48 - 1 \cdot 30) - 1 \cdot 30 \\
&= 2 \cdot 48 - 3 \cdot 30 \\
&= 2 \cdot 48 - 3 \cdot (174 - 3 \cdot 48) \\
&= 11 \cdot 48 - 3 \cdot 174
\end{aligned}$$

$$s = 11, \quad t = -3$$



c)

$$
\begin{aligned}
296 &= 1 \cdot 232 + 64 \\
232 &= 3 \cdot 64 + 40 \\
64 &= 1 \cdot 40 + 24 \\
40 &= 1 \cdot 24 + 16 \\
24 &= 1 \cdot 16 + 8 \\
16 &= 2 \cdot 8 + 0
\end{aligned}
$$

$$
\begin{aligned}
8 &= 24 - 1 \cdot 16 \\
16 &= 40 - 1 \cdot 24 \\
24 &= 64 - 1 \cdot 40 \\
40 &= 232 - 3 \cdot 64 \\
64 &= 296 - 1 \cdot 232
\end{aligned}
$$

$$
\begin{aligned}
8 &= 24 - 1 \cdot (40 - 1 \cdot 24) \\
&= 2 \cdot 24 - 1 \cdot 40 \\
&= 2 \cdot (64 - 1 \cdot 40) - 1 \cdot 40 \\
&= 2 \cdot 64 - 3 \cdot 40 \\
&= 2 \cdot 64 - 3 \cdot (232 - 3 \cdot 64) \\
&= 11 \cdot 64 - 3 \cdot 232 \\
&= 11 \cdot (296 - 1 \cdot 232) - 3 \cdot 232 \\
&= 11 \cdot 296 - 14 \cdot 232
\end{aligned}
$$

$$s = -14, \quad t = 11$$

Problem 4.6

because m and n are made allredy decomosed into primes we have to find the biges number composed of primes number that is divisible to both m and n wich will be $\gcd(m, n) = 2^5 11^8 19^5$ because we take the min power of the prime rom each number

Problem 4.7

a)

$$[14] + [9] = [23 \mod 19] = [4]$$

b)

$$[15] - [13] = [2]$$

c)

$$[8] + [12] = [96 \mod 19 = 1] = [1]$$

d) From part c)

$$[12]^{-1} = [8]$$

e) From d)

f)

$$[9][12]^{-1} = [9][8] = [72 \mod 19] = [15]$$

g)

$$[15] - [18]^{-1} = [15] - [18] = [-3] = [19 - 3] = [16]$$

Problem 4.8

a)

$$[14] + [9] = [23 \mod 20] = [3]$$

b)

$$[15] - [13] = [2]$$

c)

$$[8] \cdot [12] = [96 \bmod 20] = [16]$$

d)

$$\gcd(12, 20) = 4 \neq 1 \implies [12]^{-1} \text{ does not exist}$$

e)

$$\text{Since } [12]^{-1} \text{ does not exist, } [9][12]^{-1} \text{ is undefined}$$

f)

$$[15] - [18]^{-1} = [15] - [19] = [-4] = [20 - 4] = [16]$$

Problem 4.9

a)

$$59 = 2 * 29 + 1$$

$$1 = 59 + (-2) * 29$$

$$-2 + 59 = 57$$

$$57$$

b)

$$29^{-1} \equiv 29^{57} \quad (\mod 59)$$

$$29^{57} \equiv 841^{29} \equiv 707,281^{15} \equiv 25^{15} \equiv 625^{8} \equiv 35^{8} \equiv 1225^{4} \equiv$$

$$\equiv 45^{4} \equiv 2025^{2} \equiv 16^{2} \equiv 256 \equiv 20 \qquad (\text{mod} 59)$$

c)

$$29^{243} \bmod 13 = (29 \mod 13)^{243} \mod 13$$

By Fermat litle therem

$$3^{12} \equiv 1 \mod 13$$

$$243 = 20 * 12 + 3$$

$$29^{243} \bmod 13 = 3^{3} \bmod 13 = 27 \bmod 13 = 1$$

Problem 4.10

a) $\varphi(560) = \varphi(2^4 \cdot 5 \cdot 7) = 560 \cdot \frac{1}{2} \cdot \frac{4}{5} \cdot \frac{6}{7} = 192$

b) $\varphi(675) = \varphi(3^3 \cdot 5^2) = 675 \cdot \frac{2}{3} \cdot \frac{4}{5} = 360$

c) $\varphi(1720) = \varphi(2^3 \cdot 5 \cdot 43) = 1720 \cdot \frac{1}{2} \cdot \frac{4}{5} \cdot \frac{42}{43} = 672$

Problem 4.11

Commands:
A B swap the letters A and B
lock A B T lock letters (shown in red, can't be swapped)
unlock unlock all letters
reset restore the original string (and unlock everything)
quit exit
> Y F
YG LTEKTZL STFZ FGK TBETHZOGYQS LZXRTYZL FKGD LGFZVQKT TYUOYT-
TKOYU AYGVY QL FQF FKGD XZD
> Q A
YG LTEKTZL STFZ FGK TBETHZOGYAS LZXRTYZL FKGD LGFZVAKT TYUOYT-
TKOYU QYGVY AL FAF FKGD XZD
> L T
YG TLEKLZT SLFZ FGK LBELHZOGYAS TZXRLYZT FKGD TGFZVAKL LYUOYL-
LKOYU QYGVY AT FAF FKGD XZD
> L E
YG TELKEZT SEFZ FGK EBLEHZOGYAS TZXREYZT FKGD TGFZVAKE EYUOY-
EEKOYU QYGVY AT FAF FKGD XZD
> Y I
IG TELKEZT SEFZ FGK EBLEHZOGIAS TZXREIZT FKGD TGFZVAKE EIUOIEEKOIU
QIGVI AT FAF FKGD XZD
> U G
IU TELKEZT SEFZ FUK EBLEHZOUIAS TZXREIZT FKUD TUFZVAKE EIGOIEEKOIG
QIUVI AT FAF FKUD XZD
> O E
IU TOLKOZT SOFZ FUK OBLOHZEUIAS TZXROIZT FKUD TUFZVAKO OIGEIOOKEIG
QIUVI AT FAF FKUD XZD
> O E
IU TELKEZT SEFZ FUK EBLEHZOUIAS TZXREIZT FKUD TUFZVAKE EIGOIEEKOIG
QIUVI AT FAF FKUD XZD
> I N
NU TELKEZT SEFZ FUK EBLEHZOUNAS TZXRENZT FKUD TUFZVAKE EN-
GONEEKONG QNUVN AT FAF FKUD XZD
> O E
NU TOLKOZT SOFZ FUK OBLOHZEUNAS TZXRONZT FKUD TUFZVAKO ONGENOOKENG
QNUVN AT FAF FKUD XZD



> O I
NU TILKIZT SIFZ FUK IBLIHZEUNAS TZXRINZT FKUD TUFZVAKI INGENIIKENG QNUVN AT FAF FKUD XZD
> K R
NU TILRIZT SIFZ FUR IBLIHZEUNAS TZXKINZT FRUD TUFZVARI INGENIIRENG QNUVN AT FAF FRUD XZD
> E I
NU TELREZT SEFZ FUR EBLEHZIUNAS TZXKENZT FRUD TUFZVARE ENGINEERING QNUVN AT FAF FRUD XZD
> U O
NO TELREZT SEFZ FOR EBLEHZIONAS TZXKENZT FROD TOFZVARE ENGINEERING QNOVN AT FAF FROD XZD
> D M
NO TELREZT SEFZ FOR EBLEHZIONAS TZXKENZT FROM TOFZVARE ENGINEERING QNOVN AT FAF FROM XZM
> lock E N G I N E R I N G F A F F R O M
NO TELREZT SEFZ FOR EBLEHZIONAS TZXKENZT FROM TOFZVARE ENGINEERING QNOVN AT FAF FROM XZM
> X U
NO TELREZT SEFZ FOR EBLEHZIONAS TZUKENZT FROM TOFZVARE ENGINEERING QNOVN AT FAF FROM UZM
> Z T
NO ZELRETZ SEFT FOR EBLEHTIONAS ZTUKENTZ FROM ZOFTVARE ENGINEERING QNOVN AZ FAF FROM UTM
> Z S
NO SELRETS ZEFT FOR EBLEHTIONAZ STUKENTS FROM SOFTVARE ENGINEERING QNOVN AS FAF FROM UTM
> V W
NO SELRETS ZEFT FOR EBLEHTIONAZ STUKENTS FROM SOFTWARE ENGINEERING QNOWN AS FAF FROM UTM
> lock S T W
NO SELRETS ZEFT FOR EBLEHTIONAZ STUKENTS FROM SOFTWARE ENGINEERING QNOWN AS FAF FROM UTM
> Z L
NO SEZRETS LEFT FOR EBZEHTIONAL STUKENTS FROM SOFTWARE ENGINEERING QNOWN AS FAF FROM UTM
> K D
NO SEZRETS LEFT FOR EBZEHTIONAL STUDENTS FROM SOFTWARE ENGINEERING QNOWN AS FAF FROM UTM
> lock U D
NO SEZRETS LEFT FOR EBZEHTIONAL STUDENTS FROM SOFTWARE ENGINEERING QNOWN AS FAF FROM UTM
> Q K
NO SEZRETS LEFT FOR EBZEHTIONAL STUDENTS FROM SOFTWARE ENGINEERING KNOWN AS FAF FROM UTM
> lock K
NO SEZRETS LEFT FOR EBZEHTIONAL STUDENTS FROM SOFTWARE ENGI-



NEERING KNOWN AS FAF FROM UTM
> H P
NO SEZRETS LEFT FOR EBZEPTIONAL STUDENTS FROM SOFTWARE ENGINEERING KNOWN AS FAF FROM UTM
> B X
NO SEZRETS LEFT FOR EXZEPTIONAL STUDENTS FROM SOFTWARE ENGINEERING KNOWN AS FAF FROM UTM
> Z C
NO SECRETS LEFT FOR EXCEPTIONAL STUDENTS FROM SOFTWARE ENGINEERING KNOWN AS FAF FROM UTM
> lock C L X
NO SECRETS LEFT FOR EXCEPTIONAL STUDENTS FROM SOFTWARE ENGINEERING KNOWN AS FAF FROM UTM
> quit

NO SECRETS LEFT FOR EXCEPTIONAL STUDENTS FROM SOFTWARE ENGINEERING KNOWN AS FAF FROM UTM

```cpp
#include <iostream>
#include <sstream>
#include <string>
#include <cctype>
using namespace std;

const string RED = "\033[31m";
const string RESET_COLOR = "\033[0m";

bool locked[256] = { false };

// Print the string, showing locked letters in red
void printString(const string& s) {
    for (char c : s) {
        if (locked[(unsigned char)c])
            cout << RED << c << RESET_COLOR;
        else
            cout << c;
    }
    cout << endl;
}

void clearLocks() {
    for (int i = 0; i < 256; i++) locked[i] = false;
}

int main() {
    string original, current;

```



```
30      original = "FG LTEKTZL STYZ YGK TBETHZOGFQS LZXRTFZL YKGD
            LGYZVQKT TFUOFTTKOFU AFGVF QL YQY YKGD XZD";
31      current = original;
32
33      cout << "Commands:\n"
34           << "  A B        swap the letters A and B\n"
35           << "  lock A B T lock letters (shown in red, can't be
                swapped)\n"
36           << "  unlock     unlock all letters\n"
37           << "  reset      restore the original string (and
                unlock everything)\n"
38           << "  quit       exit\n";
39
40      string line;
41      while (cout << "> " && getline(cin, line)) {
42          istringstream iss(line);
43          string cmd;
44          if (!(iss >> cmd)) continue;
45
46          string lower = cmd;
47          for (char& c : lower) c = tolower((unsigned char)c);
48
49          if (lower == "quit") {
50              break;
51          }
52
53          if (lower == "reset") {
54              current = original;
55              clearLocks();
56              printString(current);
57              continue;
58          }
59
60          if (lower == "unlock") {
61              clearLocks();
62              printString(current);
63              continue;
64          }
65
66          if (lower == "lock") {
67              string letters;
68              while (iss >> letters) {
69                  locked[(unsigned char)letters[0]] = true;
70              }
71              printString(current);
72              continue;
73          }
74
75          // Otherwise: swap command "A B"
76          string second;
```



```
        if (!(iss >> second)) {
            cout << "Please type two letters, e.g. A B\n";
            continue;
        }

        char a = cmd[0], b = second[0];

        if (locked[(unsigned char)a] || locked[(unsigned char)b
           ]) {
            cout << "Can't swap: "
                << (locked[(unsigned char)a] ? a : b)
                << " is locked.\n";
            printString(current);
            continue;
        }

        for (char& c : current) {
            if (c == a)      c = b;
            else if (c == b) c = a;
        }

        printString(current);
    }

    return 0;
}
```

Problem 4.12

ANONYMOUS FAF-24* ARE THE BEST

```
#include <iostream>
#include <string>

using namespace std;

bool isPrime(int n) {
    if (n < 2) return false;
    if (n % 2 == 0) return n == 2;
    for (int d = 3; (long long)d * d <= n; d += 2)
        if (n % d == 0) return false;
    return true;
}

int digitSum(int n) {
    int s = 0;
    while (n > 0) { s += n % 10; n /= 10; }
    return s;
}
```



```
19
20
21  string divide(const string& num, int d, int& rem) {
22      string q;
23      long long r = 0;
24      for (char ch : num) {
25          r = r * 10 + (ch - '0');
26          q += char('0' + r / d);
27          r %= d;
28      }
29      rem = (int)r;
30      size_t pos = q.find_first_not_of('0');
31      return pos == string::npos ? "0" : q.substr(pos);
32  }
33
34  char decodeChar(int code) {
35      if (code >= 1 && code <= 26) return char('A' + code - 1);
36      switch (code) {
37      case 27: return ' ';
38      case 28: return '*';
39      case 29: return ')';
40      case 30: return '?';
41      case 31: return ':';
42      case 32: return '(';
43      case 33: return '-';
44      case 34: return '+';
45      case 35: return '!';
46      case 36: return '1';
47      case 37: return '2';
48      case 38: return '3';
49      case 39: return '4';
50      case 40: return '5';
51      case 41: return '6';
52      case 42: return '7';
53      case 43: return '8';
54      case 44: return '9';
55      case 45: return '0';
56      }
57      return 0;
58  }
59
60
61  string decodeMessage(const string& digits, string& leftover) {
62      string best;
63      int bestValid = -1;
64      for (int offset = 0; offset < 2; ++offset) {
65          string s = (offset ? "0" : "") + digits;
66          string text;
67          int valid = 0;
68          for (size_t i = 0; i + 1 < s.size(); i += 2) {
```



```
            int code = (s[i] - '0') * 10 + (s[i + 1] - '0');
            char c = decodeChar(code);
            if (c) { text += c; ++valid; }
            else text += '#';
        }
        if (valid > bestValid) {
            bestValid = valid;
            best = text;
            leftover = (s.size() % 2) ? string(1, s.back()) : "
               ";
        }
    }
    return best;
}

int main() {
    const string cipher =
        "
           114079623885113465875798552685536069236148464919596369023295391
           ";

    int largest = -1, key = -1;
    string plainDigits;
    for (int n = 999999; n >= 100000; --n) {
        if (digitSum(n) != 38 || !isPrime(n)) continue;
        if (largest < 0) largest = n;
        int rem;
        string q = divide(cipher, n, rem);
        if (rem == 0) { key = n; plainDigits = q; break; }
    }
    if (key < 0) { cerr << "No key divides the ciphertext\n";
       return 1; }

    cout << "Cipher length                        : " << cipher.
       size() << " digits\n";
    cout << "Largest 6-digit prime, digit sum 38 : " << largest
        << "\n";
    cout << "Key that divides the ciphertext     : " << key <<
       "\n\n";
    cout << "Decrypted number (c / k):\n" << plainDigits << "\n
       \n";

    string leftover;
    string text = decodeMessage(plainDigits, leftover);
    cout << "Decoded message:\n" << text << "\n";
    if (!leftover.empty())
        cout << "(one unpaired trailing digit: " << leftover <<
            ")\n";
    return 0;
}
```



Problem 4.13

The key is $k = 1013$ because $c_1 = km_1$ and $c_2 = km_2$ so $k = \frac{\gcd(c_1,c_2)}{\gcd(m_1,m_2)}$

RECTOR UTMM
UTM FCIM 3-204

Code:

```
    #include <iostream>
#include <string>

using namespace std;

string divide(const string& num, long long d, long long& rem) {
    string q;
    long long r = 0;
    for (char ch : num) {
        r = r * 10 + (ch - '0');
        q += char('0' + r / d);
        r %= d;
    }
    rem = r;
    size_t pos = q.find_first_not_of('0');
    return pos == string::npos ? "0" : q.substr(pos);
}

char decodeChar(int code) {
    if (code >= 1 && code <= 26) return char('A' + code - 1);
    switch (code) {
    case 27: return ' ';
    case 28: return '*';
    case 29: return ')';
    case 30: return '?';
    case 31: return ':';
    case 32: return '(';
    case 33: return '-';
    case 34: return '+';
    case 35: return '!';
    case 36: return '1';
    case 37: return '2';
    case 38: return '3';
    case 39: return '4';
    case 40: return '5';
    case 41: return '6';
    case 42: return '7';
    case 43: return '8';
    case 44: return '9';
    case 45: return '0';
```



```
41      }
42      return 0;
43  }
44
45  string decodeMessage(const string& digits, string& leftover,
        int& valid, int& total) {
46      string best;
47      int bestValid = -1;
48      for (int offset = 0; offset < 2; ++offset) {
49          string s = (offset ? "0" : "") + digits;
50          string text;
51          int v = 0;
52          for (size_t i = 0; i + 1 < s.size(); i += 2) {
53              int code = (s[i] - '0') * 10 + (s[i + 1] - '0');
54              char c = decodeChar(code);
55              if (c) { text += c; ++v; }
56              else text += '#';
57          }
58          if (v > bestValid) {
59              bestValid = v;
60              best = text;
61              valid = v;
62              total = (int)(s.size() / 2);
63              leftover = (s.size() % 2) ? string(1, s.back()) : "
                    ";
64          }
65      }
66      return best;
67  }
68
69  int main(int argc, char* argv[]) {
70      string cipher, keyStr;
71      if (argc >= 3) {
72          cipher = argv[1];
73          keyStr = argv[2];
74      }
75      else {
76          cout << "Ciphertext: ";
77          cin >> cipher;
78          cout << "Key:        ";
79          cin >> keyStr;
80      }
81
82      for (char c : cipher + keyStr)
83          if (c < '0' || c > '9') { cerr << "Only digits are
                allowed.\n"; return 1; }
84      long long key = stoll(keyStr);
85      if (key < 2) { cerr << "The key must be at least 2.\n";
            return 1; }
86
```

```
87      long long rem;
88      string plainDigits = divide(cipher, key, rem);
89      if (rem != 0) {
90          cout << "Key " << key << " does not divide the
                ciphertext (remainder "
91               << rem << "), so it is not the right key.\n";
92          return 2;
93      }
94
95      string leftover;
96      int valid = 0, total = 0;
97      string text = decodeMessage(plainDigits, leftover, valid,
            total);
98
99      cout << "\nDecrypted number: " << plainDigits << "\n";
100     cout << "Decoded message : " << text << "\n";
101     if (!leftover.empty())
102         cout << "(one unpaired trailing digit: " << leftover <<
                ")\n";
103     if (valid != total)
104         cout << "(only " << valid << " of " << total
105         << " two-digit codes are valid, so this key is probably
                wrong)\n";
106     return 0;
107 }
```

Problem 4.14

a)

$$m^* = rem(m \cdot k, p)$$

$$m^* = \text{rem}(475 \cdot 127,\ \ 503) = rem(60.325,\ \ 503) = 468$$

$$m^* = 468$$

b)

$$m = \text{rem}(m^* \cdot k^{-1}, p) = \text{rem}(443 \cdot 6^{-1}, 503)$$

$$503 = 83 \cdot 6 + 5 \implies 5 = 503 - 83 \cdot 6$$

$$6 = 1 \cdot 5 + 1 \implies 1 = 6 - 1 \cdot 5$$

$$1 = 84 \cdot 6 - 1 \cdot 503$$

$$6^{-1} \equiv 84 \pmod{503}$$

$$m = \text{rem}(443 \cdot 84, 503) = \text{rem}(37.212,\ 503) = 493$$

$$m = 493$$



Problem 4.15

$$
\begin{aligned}
k &= \text{rem}(m^{p-2} \cdot m^*, p) \\
&= \text{rem}(8^{11} \cdot 4, 13) \\
&= \text{rem}(8^{-1} \cdot 4, 13) \quad \text{where } 8 \cdot 5 \equiv 1 \pmod{13} \implies 8^{-1} = 5 \\
&= \text{rem}(5 \cdot 4, 13) \\
&= \text{rem}(20, 13) = 7
\end{aligned}
$$